

Source: <https://hypermode.com/blog/graph-partitioning>
<https://github.com/MickaelBergem/SpectralPartitionAlgorithm>
<https://stackoverflow.com/questions/59116628/algorithm-to-partition-graph-into-complete-subgraphs>

I. Written report

1. Project Overview

- Dataset: Astrophysic Collaboration Network - <https://snap.stanford.edu/data/ca-GrQc.html>
- This network contains collaborations between authors' papers submitted to Astro Physics. The goal is to find best k representations as potential subfields we can divide this network of collaboration into.

2. Data processing

- I did not do any extra data cleaning. I only deleted unnecessary text from the data file in the header so it is easier for me to load the data file later on.
- I use io File to open the data file in rust.

3. Code structure

a. Module - 2 main module

- Graph.rs: defining graph structure to store the dataset of nodes and edges. Responsible for parsing the dataset file into the structure memory using function load_data - io file.
- Partition.rs: I downloaded ndarray and ndarray - linalg to use linalg function like calculating eigenvalue and eigenvector. Implementing graph-partitioning algorithm using spectral bisection and recursive splits. Also implementing a function to calculate the average degree and edgecut.

b. Key functions & types

```
src > graph.rs > load_data
1 use std::collections::HashMap;
2 use std::error::Error;
3 use std::fs::File;
4 use std::io::{self, BufRead};
5
6 //creating graph structure contains list of node ID and undirected edges
7 0 implementations
8 pub struct Graph{
9     pub nodes: Vec<String>, //map each author ID to a base index
10     _idx: HashMap<String,usize>, // lookup table to avoid duplicate
11     pub edges: Vec<(usize,usize)>, //index pairs
12 }
13
14 //load file into the graph
15 pub fn load_data(path:&str) -> Result<Graph, Box<dyn Error>> {
16     let file: File = File::open(path)?;
17     let reader: BufReader<File> = io::BufReader::new(inner: file);
18     // create empty Vec and HashMap to store data
19     let mut nodes: Vec<String> = Vec::new();
20     let mut idx: HashMap<String, usize> = HashMap::new();
21     let mut edges: Vec<(usize, usize)> = Vec::new();
22
23     for lines: Result<String, Error> in reader.lines(){
24         let line: String = lines?;
25         let parts: Vec<String> = line.trim().split_whitespace().collect(); // trimming the whitespace between value
26         // convert first and second node of part into string
27         let x: String = parts[0].to_string();
28         let y: String = parts[1].to_string();
29
30         //either assign into node if it is not there yet or look up index and return index
31         let idx_x: usize = *idx.entry(key: x.clone()).or_insert_with(default: || {nodes.push(x.clone()); nodes.len()-1});
32         let idx_y: usize = *idx.entry(key: y.clone()).or_insert_with(default: || {nodes.push(y.clone()); nodes.len()-1});
33
34         edges.push((idx_x,idx_y));
35     }
36     Ok(Graph(nodes, _idx: idx, edges))
37 }
```

- Struct: defining structure for graph. Storing nodes and vectors of string and edges as vectors of node indices - store it as vec so it is easier to retrieve . Index is a hashmap as a lookup table.
- Fn load_data: using io file to load the dataset file and open it. Once it opens we do some clean up the whitespace. Then we push the node into vec and node indices into edges to create our graph based on struct above.

```
fn spectral_bisect(
    graph: &Graph,
    subset: &[usize],
) -> (Vec<usize>, Vec<usize>){ // return a matrix of vec and vec
    let n: usize = subset.len();
    // Basically in here, you create a Laplacian matrix - represent of the graph that contains all the connectivity
    let mut lap: ArrayBase<OwnedRepr<f64>, ...> = Array2::<f64>::zeros(shape: (n,n)); //create a matrix with n x n
    let mut index_map: HashMap<usize, usize> = HashMap::with_capacity(n);
    for (i: usize, &node: usize) in subset.iter().enumerate(){
        index_map.insert(k: node,v: i);
    }
    for &(u: usize,v: usize) in &graph.edges{ // for each edge in the graph
        if let (Some(i: usize), Some(j: usize)) = (index_map.get(&u).copied(),index_map.get(&v).copied()){
            lap[(i,j)] += 1.0;
            lap[(j,i)] += 1.0;
        }
    }
}

// computing unnormalized graph Laplacian
for g: usize in 0..n{ // for each node
    let degree: f64 = lap.row(index: g).iter().sum(); // computing the node's degree by sum of its adjacency row
    lap[(g,g)] = degree;
    for f: usize in 0..n{ // for every other column f is different than g
        if g != f{
            lap[(g,f)] = -lap[(g,f)]; // negate it
        }
    }
}

let (_vals: ArrayBase<OwnedRepr<f64>, ...>, vecs: ArrayBase<0...>) = lap.eigh(UPLO: Lower).unwrap();
let fielder: ArrayBase<OwnedRepr<f64>, ...> = vecs.column(index: 1).to_owned(); // taking second smallest eigenvector (idx = 1)
//each node in subset pair with fielder vector and sort
let mut pairs: Vec<(usize, f64)> = subset.iter().enumerate().map(|(i: usize, &node: usize)| (node, fielder[i])).collect();
pairs.sort_by(compare: |a: &(usize, f64), b: &(usize, f64)| a.1.partial_cmp(&b.1).unwrap());
// splitting into 2 partitions
let mid: usize = pairs.len() / 2;
(
    pairs[..mid].iter().map(|&(node: usize,_)| node).collect(),
    pairs[mid..].iter().map(|&(node: usize,_)| node).collect(),
)
} fn spectral_bisect
```

- Fn spectral_bisect: takes a subset of nodes, builds its un-normalized Laplacian matrix (by counting edges among those nodes and setting diagonal entries equal to each node's degree), then uses an eigen-decomposition to extract the second-smallest eigenvector- Fiedler vector. It pairs each node with its Fiedler value, sorts them by that value, and finally splits the sorted list in half, producing two groups that approximate a balanced, low-cut spectral partition.

```

46 //Recursively partition node in k cluster by performing spectral bisection
47 pub fn graph_partition(
48     graph: &Graph, // calling the graph
49     k: usize, //desired number of partitions - subfields of research
50 ) -> Vec<Vec<usize>>{ // return vector of listing the node-indices in that partition.
51     let mut parts: Vec<Vec<usize>> = vec![0; graph.nodes.len()]; // start with first part containing all the node indices.
52     while parts.len() < k { // calling recursively
53         let index: usize = parts.iter().enumerate().max_by_key(|(_,p): &&Vec<usize>| p.len()).unwrap().0; // pick the largest current part to split
54         let subset: Vec<usize> = parts.remove(index); //remove it from the list
55         let (p1: Vec<usize>, p2: Vec<usize>) = spectral_bisect(graph, &subset); //bisect the subset into two smaller subsets
56         parts.push(p1); // add those two new subsets back into the list
57         parts.push(p2);
58     }
59     parts // return parts
60 }

```

- Fn graph_partition: function that graph and k - number of desired subfield- as input. Here the function basically starts one graph then splits it into 2 via fn spectral_bisect. It is repeatedly splitting until reaching k number of subfields.

```

// compute a vector of degrees for each node index
fn degree_distribution(graph: &Graph) -> Vec<usize> {
    let mut degree: Vec<usize> = vec![0; graph.nodes.len()]; // create a vector of zeroes with n length
    for (u: usize, v: usize) in &graph.edges { // for each tuple of edge in graph, increase degree by 1
        degree[u] += 1;
        degree[v] += 1;
    }
    degree
}

//compute average degree
pub fn avg_degree(graph: &Graph) -> f64 { // taking the average of all the degree of nodes above to get the average
    let deg: Vec<usize> = degree_distribution(graph);
    let sum: usize = deg.iter().sum();
    sum as f64 / graph.nodes.len() as f64
}

```

```

76 // calculate number of edges cross the subfields
77 pub fn edge_cut(graph: &Graph, parts:&[Vec<usize>]) -> usize{
78     let mut idx: Vec<usize> = vec![0; graph.nodes.len()]; // create vector of zeros with n length of node
79     for (i: usize, subset: &Vec<usize>) in parts.iter().enumerate(){ // for each i as partition number and subset - vector of node indices
80         for &node: usize in subset{ // for each node in subset
81             idx[node] = i; // set the index of node = i so we know which part is in
82         }
83     }
84     // return count of edges that have endpoints in different part
85     graph.edges.iter().filter(|&(u: usize, v: usize)| idx[u] != idx[v]).count()
86 }
87

```

- Fn degree_distribution & avg_degree: computes how “connected” the graph is on average by first tallying each node’s degree (number of incident edges) via degree_distribution, summing those degrees, and then dividing by the total number of nodes.
- Fn edge_cut: counting every edge whose endpoints lie in different parts. It first builds a quick lookup from each node to its assigned partition, then filters the full edge list to those that cross a partition boundary. The returned the total number of inter-partition edges.

c. Main workflow

```

mod graph;
mod partition;
use std::error::Error;
// calling each public function in each module
fn main() -> Result<(), Box<dyn Error>> {
    let graph = graph::load_data("CA-GrQc.txt")?;
    println!("Graph has {} nodes and {} edges", graph.nodes.len(), graph.edges.len());
}

```

```

let parts = partition::graph_partition(&graph, 11);
for (i, subset) in parts.iter().enumerate(){
    println!("Subfield {:>2} : {} authors ", i +1, subset.len());
}
let average_degree = partition::avg_degree(&graph);
println!("Average degree: {:.2}", average_degree);
let edge_cut= partition::edge_cut(&graph, &parts);
println!("edge-cut:{}", edge_cut);
Ok(())
}

```

- For my main, I basically call each public function from each module to use. I called load_data to load my dataset into a graph. Then I used all public functions in module partition.rs to proceed my partition graph and statistics related.
d. Test

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
● [*] Executing task: cargo test --package DS210_project --bin DS210_project -- partition::test::test_partition --exact --show-output

Finished `test` profile [unoptimized + debuginfo] target(s) in 0.16s
Running unittests src/main.rs (target/debug/deps/DS210_project-b15f082a8e39116c)

running 1 test
test partition::test::test_partition ... ok

successes:
successes:
partition::test::test_partition

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out; finished in 0.01s

[*] Terminal will be reused by tasks, press any key to close it.
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
● Finished `test` profile [unoptimized + debuginfo] target(s) in 0.57s
Running unittests src/main.rs (target/debug/deps/DS210_project-b15f082a8e39116c)

running 1 test
test graph::test::test_load_data ... ok

successes:
successes:
graph::test::test_load_data

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out; finished in 0.01s

[*] Terminal will be reused by tasks, press any key to close it.
```

- I ran two tests on load_data and graph_partition. For load_data, I ran to test out if it reads my file correctly and gives the correct output of edges as well as nodes. For graph_partition, I ran a test to see if it split my test into the right number of k inputs.

e. Results

```

Graph has 5242 nodes and 28980 edges
Subfield 1 : 655 authors
Subfield 2 : 655 authors
Subfield 3 : 655 authors
Subfield 4 : 655 authors
Subfield 5 : 655 authors
Subfield 6 : 328 authors
Subfield 7 : 328 authors
Subfield 8 : 328 authors
Subfield 9 : 328 authors
Subfield 10 : 327 authors
Subfield 11 : 328 authors
Average degree: 11.06
edge-cut:10134
trangnguyen@crc-dot1x-nat-10-239-175-101 DS210_project %

```

- Average degree: this shows that on average, each author or researcher would have 11 collaborations.
- Edge-cut: this shows that ~35% span cluster boundaries and ~65% capture within clusters.
- After trying several different values of k , I decided $k = 11$ is the best representation for this graph because given the statistic from the data set, clustering coefficient is ~ 0.53 from the website, which indicates that this dataset has moderate clustering. By choosing $k = 11$, we still have enough subfields that capture 65% of collaboration without being under splitting that misses detailed information or over-splitting.
- Why is this important to determine the subfield? It is essential to see the co-author of research or communities to identify if authors or researchers have the same theory or results of a certain area that they are interested in. It also helps to see what are some main sub-area in astrophysics that are becoming more interesting or getting more attention, from there we can see each subfield how it evolves and grows over time as a smaller community.

f. Usage Information

- To run my code: `cargo run` or `cargo run --release`

II. Codes comment

graph.rs

```

use std::collections::HashMap;
use std::error::Error;
use std::fs::File;
use std::io::{self, BufRead};

//this module is for loading the dataset into graph

//creating graph structure contains list of node ID and undirected edges

```

```

pub struct Graph{
    pub nodes: Vec<String>, //map each author ID to a base index
    _idx: HashMap<String,usize>, // lookup table to avoid duplicate
    pub edges: Vec<(usize,usize)>, //index pairs
}

//load file into the graph
pub fn load_data(path:&str) -> Result<Graph, Box<dyn Error>> {
    let file = File::open(path)?;
    let reader = io::BufReader::new(file);
    // create empty Vec and Hashmap to store data
    let mut nodes = Vec::new();
    let mut idx = HashMap::new();
    let mut edges = Vec::new();

    for lines in reader.lines(){
        let line = lines?;
        let parts: Vec<&str> = line.trim().split_whitespace().collect(); // trimming
the whitespace between value
        // convert first and second node of part into string
        let x = parts[0].to_string();
        let y = parts[1].to_string();

        //either assign into node if it is not there yet or look up index and return index
        let idx_x = *idx.entry(x.clone()).or_insert_with(|| {nodes.push(x.clone());
nodes.len()-1});
        let idx_y = *idx.entry(y.clone()).or_insert_with(|| {nodes.push(y.clone());
nodes.len()-1});

        edges.push((idx_x,idx_y));
    }
    Ok(Graph{nodes, _idx: idx, edges})
}

#[cfg(test)]
mod test{
    use super::*;
    #[test]
    fn test_load_data() {
        let test_data = load_data("test_data_file.txt").unwrap();
        assert_eq!( test_data.nodes.len(), 3 );
        assert_eq!( test_data.edges.len(), 3);
    }
}

```

```
}
```

partition.rs

```
use std::collections::HashMap;
use ndarray::Array2;
use ndarray_linalg::{Eigh, UPLQ };
use crate::graph::Graph;

// this module is responsible for partitioning the graph repetitively as desire
fn spectral_bisect(
    graph: &Graph, // reference graph
    subset: &[usize], // input a slice of a number
) -> (Vec<usize>, Vec<usize>){ // return a matrix of vec and vec
    let n = subset.len();

    // Basically in here, you create a Laplacian matrix - represent of the graph that
    contains all the connectivity

    let mut lap = Array2::::zeros((n,n)); //create a matrix with n x n
    let mut index_map = HashMap::with_capacity(n); // create a look up map
    for (i, &node) in subset.iter().enumerate(){
        index_map.insert(node,i);
    }

    for &(u,v) in &graph.edges{ // for each edge in the graph
        if let (Some(i), Some(j)) =
(index_map.get(&u).copied(), index_map.get(&v).copied()){ //if both endpoints are in the
subset

            lap[(i,j)] += 1.0; // incese the adjency on the laplacian matrix by 1 in
both way since it is undirected

            lap[(j,i)] += 1.0;
        }
    }

    // computing unnormalized graph Laplacian
    for g in 0..n{ // for each node
        let degree: f64 = lap.row(g).iter().sum(); // computing the node's degree by
sum of its adjacency row
        lap[(g,g)] = degree;
        for f in 0..n{ // for every other column f is different than g
            if g != f{
                lap[(g,f)] = -lap[(g,f)]; // negate it
            }
        }
    }
}
```



```

    let (_vals, vecs) = lap.eigh(UPLQ::Lower).unwrap();//calling .eigh() from ndarray-
linalg to calculate eigenvalue and eigenvector of lower triangle
    let fielder = vecs.column(1).to_owned(); // taking second smallest eigenvector (idx
= 1)
    //each node in subset pair with fielder vector and sort in order
    let mut pairs: Vec<usize, f64> = subset.iter().enumerate().map(|(i, &node)|
(node, fielder[i])).collect();
    pairs.sort_by(|a,b| a.1.partial_cmp(&b.1).unwrap());
    // splitting into 2 partitions
    let mid = pairs.len() / 2;
    (
        pairs[..mid].iter().map(|&(node,_)| node).collect(),
        pairs[mid..].iter().map(|&(node,_)| node).collect(),
    )
}

//Recursively partition node in k cluster by performing spectral bisection
pub fn graph_partition(
    graph: &Graph, // calling the graph
    k: usize, //desired number of partitons - subfields of research
) -> Vec<Vec<usize>>{ // return vector of listing the node-indices in that partition.
    let mut parts: Vec<_> = vec![(0..graph.nodes.len()).collect::<Vec<_>>()]; // start
with first part containing all the node indices.
    while parts.len() < k { // calling recursively
        let index = parts.iter().enumerate().max_by_key(|(_,p)| p.len()).unwrap().0;//
pick the largest current part to split
        let subset = parts.remove(index);//remove it from the list
        let (p1,p2) = spectral_bisect(graph, &subset); //bisect the subset into two
smaller subsets
        parts.push(p1); // add those two new subsets back into the list
        parts.push(p2);
    }
    parts // return parts
}

// compute a vector of degrees for each node index
fn degree_distribution(graph: &Graph) -> Vec<usize> {
    let mut degree = vec![0; graph.nodes.len()]; // create a vector of zeroes with n
length
    for &(u,v) in &graph.edges { // for each tuple of edge in graph, increase degree by
1
        degree[u] +=1;
        degree[v] += 1;
    }
}

```

```

    }

    degree
}

//compute average degree
pub fn avg_degree(graph: &Graph) -> f64{ // taking the average of all the degree of
nodes above to get the average
    let deg = degree_distribution(graph);
    let sum: usize = deg.iter().sum();
    sum as f64 / graph.nodes.len() as f64
}

// calculate number of edges cross the subfields
pub fn edge_cut(graph: &Graph, parts:&[Vec<usize>]) -> usize{
    let mut idx = vec![0; graph.nodes.len()]; // create vector of zeros with n length of
node
    for (i, subset) in parts.iter().enumerate(){ // for each i as partion number and
subset - vector of node indice
        for &node in subset{ // for each node in subset
            idx[node] = i; // set the index of node = i so we know which part is in
        }
    }
    // return count of edges that have endpoints in different part
    graph.edges.iter().filter(|&(u,v)| idx[u] != idx[v]).count()
}

#[cfg(test)]
mod test{
    use super::*;
    use crate::graph::load_data;

    #[test]
    fn test_partition() {
        let graph = load_data("test_data_file.txt").unwrap();
        let parts = graph_partition(&graph, 2);
        assert_eq!(parts.len(), 2);
    }
}

```

main.rs

```

mod graph;
mod partition;
use std::error::Error;
// calling each public function in each module
fn main() -> Result<(), Box<dyn Error>> {

```

```
let graph = graph::load_data("CA-GrQc.txt");
println!("Graph has {} nodes and {} edges", graph.nodes.len(), graph.edges.len());
let parts = partition::graph_partition(&graph, 11);
for (i, subset) in parts.iter().enumerate(){
    println!("Subfield {:>2} : {} authors ", i +1, subset.len());
}

let average_degree = partition::avg_degree(&graph);
println!("Average degree: {:.2}", average_degree);
let edge_cut= partition::edge_cut(&graph, &parts);
println!("edge-cut:{}", edge_cut);

Ok(())
}
```